TraceHunt AI

From scattered signals to a reviewable incident investigation

Monitor APIs and ingest structured logs

Connect incidents with logs and deployments

Explain likely causes while keeping operators in control

TraceHunt AI is a single-workspace observability and incident intelligence application. It combines an operator dashboard, a validated ingestion API, a durable SQL event outbox, scheduled HTTP checks, incident rules, and evidence-based analysis. This guide explains the implementation in the repository, from its purpose to its operational boundaries.

The platform is a development foundation, not a qualified high-availability monitoring service. AI assistance produces hypotheses and suggested actions; it does not prove a cause or execute remediation. The examples and graphs in this document are invented teaching data, not measurements from a customer system. Implementation references are pinned to the source revision listed below.

Implementation reference: README.md

24 topics / architecture and workflow diagrams / two illustrative graphs
Prepared 23 October 2026. Read alongside the repository README and operator runbook.

The problem it solves

Engineers need context, not just another error notification

Health checks show symptoms but little explanation

Logs and release history often live in separate places

Repeated failures create noise during investigation

An API can return errors while its process remains alive. A latency spike may coincide with a release, database connection pressure, or an upstream timeout. Without a shared view, engineers manually search logs, compare timestamps, identify the affected service, and collect evidence before deciding what to do.

TraceHunt brings these signals into one workflow. It records checks and logs, creates a deduplicated incident when a rule matches, attaches relevant evidence, and presents a probable cause with recommended actions. The intended benefit is less manual context gathering and a clearer investigation record. The repository does not establish a measured reduction in downtime or mean time to recovery.

Implementation reference: docs/architecture/system-design.md

Who uses it and why

A shared investigation surface for a small engineering team

On-call engineers triage active incidents

Backend engineers search service errors and trace IDs

Release owners compare failures with deployment history

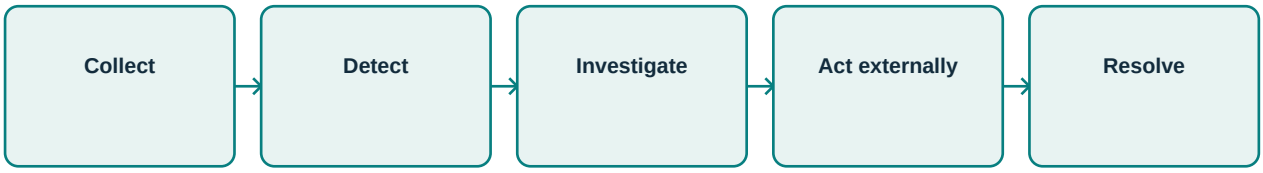
An on-call engineer starts with the overview and monitor status, opens the affected incident, checks the evidence and analysis source, and moves the incident to investigating. A backend engineer searches structured logs by service, severity, or trace ID and checks the hypothesis against the application and its dependencies. A release owner records versions and commit references so deployments can be considered during analysis.

These are workflow roles, not implemented access-control roles. The application has one shared workspace key outside demo mode, without per-user identities or tenant isolation. An audit actor is a supplied label, not proof of an authenticated person. Teams needing accountability or restricted administrative privileges must add an authentication and authorization layer before using it for sensitive workloads.

Implementation reference: SECURITY.md

The operating loop

Collect, detect, investigate, act, and record the outcome



Conceptual flow; the accompanying text defines exact relationships and optional paths.

Collect logs, checks, and deployment events

Evaluate rules and create an incident with context

Review evidence, act externally, and resolve manually

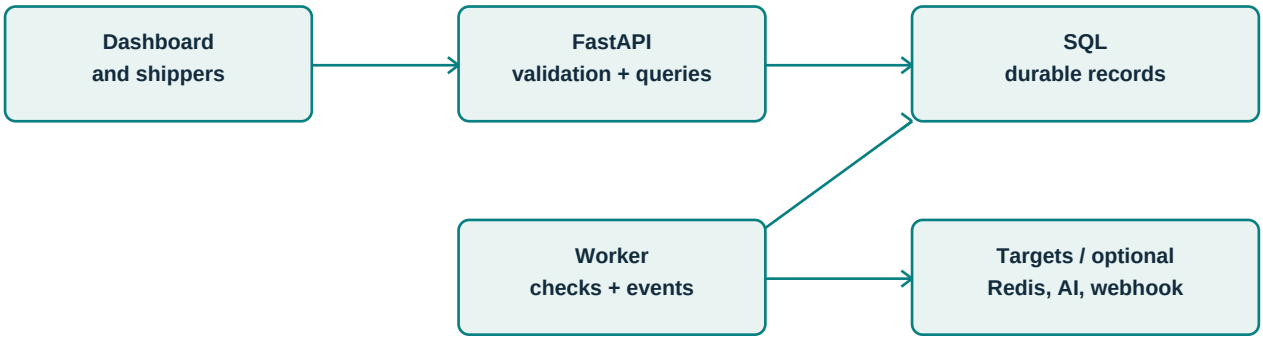
Collection is separate from investigation. Applications submit JSON logs and deployments to FastAPI. The worker checks enabled HTTP monitors and evaluates persisted events. A matching rule produces an incident, its timeline, and an incident-created event. That event triggers automatic analysis and, when configured, webhook delivery.

The operator reviews the analysis alongside raw evidence. Recommended actions may include checking a database pool, reviewing a deployment, or investigating an upstream dependency. Any mitigation happens outside TraceHunt. The operator then records status changes in the application. This human decision step matters because correlated evidence and generated explanations can be incomplete or wrong. Recovery of a monitor does not automatically close its incident.

Implementation reference: docs/architecture/system-design.md

System architecture

SQL holds the durable truth; the worker moves work forward



Next.js dashboard uses a same-origin API proxy

FastAPI and the worker share the SQL database

Redis, webhooks, and AI are optional integrations

Next.js serves the dashboard and forwards same-origin /api/v1 requests to FastAPI. Applications and log shippers can also call FastAPI directly. The API validates inputs and writes durable records. The background worker uses the same database to schedule monitors, claim outbox events, evaluate incident rules, run analysis, retry failures, and perform retention.

SQLite supports local development; PostgreSQL is used by the Compose stack. Redis Streams carries event identifiers, while SQL remains authoritative and is swept even when Redis is configured. Redis is not a direct dashboard notification channel. Optional external calls include validated HTTP probes, an incident webhook, and the OpenAI Responses API. Each creates a distinct network and operational boundary.

Implementation reference: docs/architecture/system-design.md

Technology and responsibilities

Each component has a focused job

Next.js + React: navigation and operator views

FastAPI + SQLAlchemy: validation and persistence

PostgreSQL + Redis + Docker: deployment building blocks

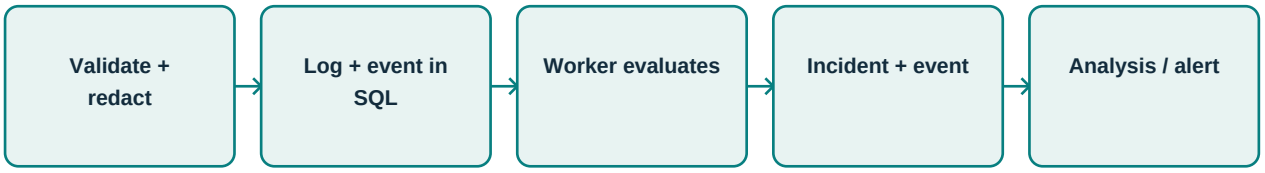
The web application lives in apps/web. The API entry point in apps/api/app/main.py exposes queries, mutations, authentication, health, metrics, and the activity stream. Business logic in services.py handles ingestion, checks, incident creation, and analysis. worker.py handles schedules, claims, retries, and cleanup. SQLAlchemy provides the database abstraction and Alembic tracks schema migrations.

The local path can run with SQLite and no Redis or provider key. The Docker Compose path separates web, API, worker, PostgreSQL, and Redis into services. These components make the application understandable and extensible, but their presence alone does not establish throughput, fault tolerance, or horizontal scalability. Those properties require deployment design and measured testing.

Implementation reference: docs/architecture/system-design.md

Structured log ingestion

An error record and its event are committed together



Conceptual flow; the accompanying text defines exact relationships and optional paths.

Validate service, severity, message, and metadata

Redact selected sensitive fields on a best-effort basis

Persist ERROR/FATAL logs with a log.error outbox event

Structured logs provide searchable service names, normalized severity, messages, optional trace IDs, and metadata. Numeric metadata.latency_ms can contribute to overview latency calculations. Timestamps are assigned at ingestion; historical source timestamps are not accepted. A trace_id links searchable records but does not create distributed tracing spans or an OpenTelemetry collector.

For ERROR and FATAL records, the database transaction inserts both the log and a log.error event. This avoids committing a log while forgetting its detection event. The worker later evaluates the service error window. Redaction reduces some common secret exposure but is regex-based and incomplete; applications should remove secrets and unnecessary personal information before sending logs.

Implementation reference: apps/api/app/services.py

HTTP monitoring

Status and header latency define the result

Unexpected status or request failure: down

Expected status with excessive latency: degraded

Expected status within threshold: up

A monitor supports GET or HEAD, one expected HTTP status, and a latency threshold. Timing ends when response headers arrive; response bodies are not downloaded. The worker schedules enabled, non-demo monitors sequentially. Intervals are scheduling targets rather than strict deadlines, and a slow target can delay other due checks. Manual checks can run on paused monitors.

URLs are revalidated at execution. DNS results are checked and pinned, redirects and environment proxies are disabled, and TLS verification stays enabled. Private-network access is blocked by default. Sample monitors return synthetic manual-check results without network requests and are skipped by the scheduler. A degraded result can still count as successful uptime because its HTTP status matched.

Implementation reference: docs/architecture/system-design.md

Incident detection rules

A transparent fixed threshold, not a learned anomaly model

Default error rule: 5 errors in the last 300 seconds

New error incidents: high; critical at 3 times threshold

Active fingerprints suppress repeated incident creation

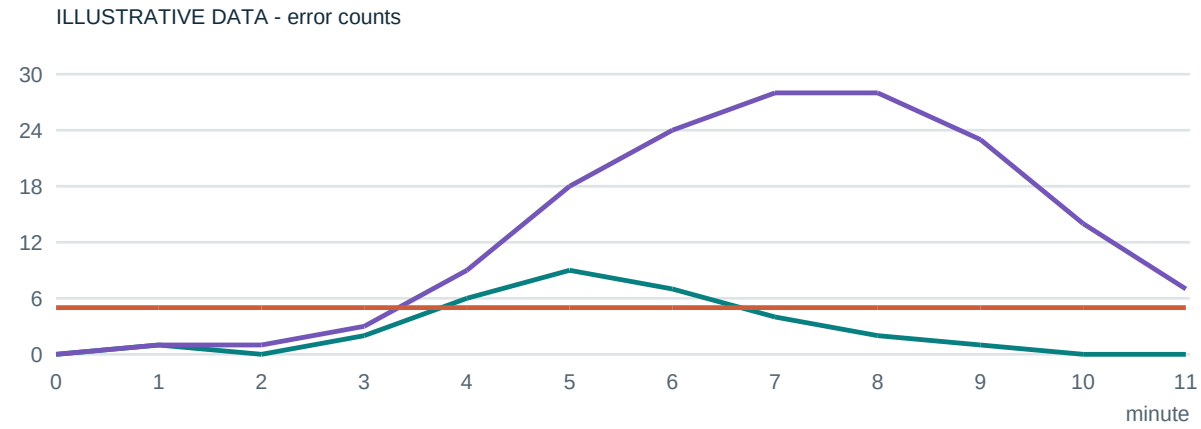
The worker counts recent ERROR/FATAL logs for an exact service at processing time. Five errors in the default 300-second window meet the error-burst rule; three times the threshold makes a newly created incident critical. A rule-state row serializes creation and stores the last incident reference. Monitor rules use a fingerprint per monitor: down creates critical severity and degraded creates medium severity.

An active incident suppresses duplicates. Repeated matching events do not raise its severity or append every log to its timeline. After resolution, a default 900-second cooldown is measured from the previous incident creation time. Delayed processing can miss an old burst after it leaves the current window. These are explicit rule semantics, not adaptive baselines or guaranteed event-time detection.

Implementation reference: docs/architecture/system-design.md

Reading the error graph

Illustrative data: a burst crosses the detection threshold



Compare errors per minute with the rolling count

Five-minute rolling count crosses 5 at minute 4

A later larger burst does not upgrade an active incident

The graph uses invented error counts per minute: 0, 1, 0, 2, 6, 9, 7, 4, 2, 1, 0, 0. The second line sums each minute and the four preceding minutes, with earlier values assumed zero. This is a discrete illustration of the default five-minute rule, evaluated at the end of each minute, not an exact replay of the asynchronous worker.

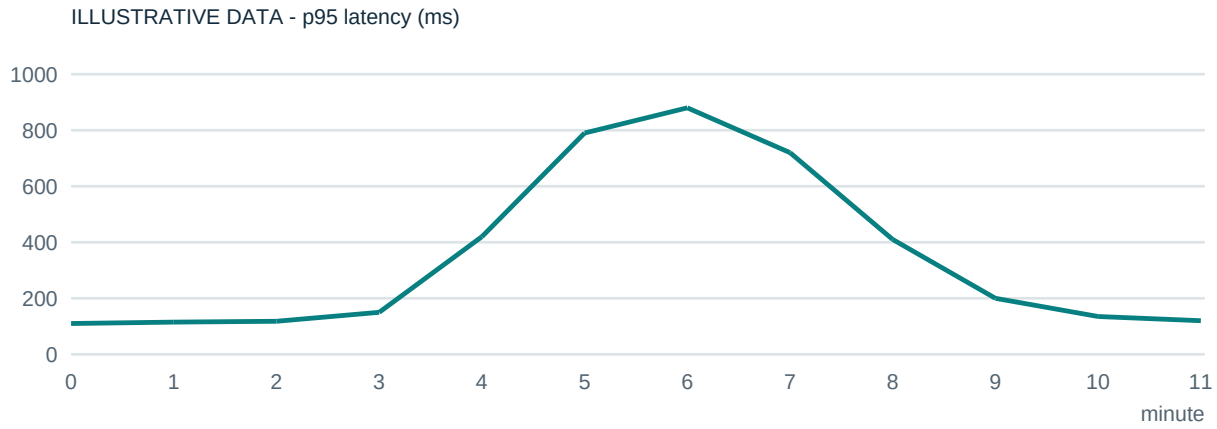
At minute 4 the rolling count reaches 9, enough to open a high incident. At minute 5 it reaches 18, above the critical creation threshold of 15. Because an incident is already active, the implementation does not automatically upgrade it. Counts falling afterward do not resolve it. No values here represent actual project traffic, accuracy, or measured operational performance.

Implementation reference: docs/architecture/system-design.md

Legend: teal = errors/minute; purple = rolling five-minute count; orange = threshold 5.

Reading the latency graph

Illustrative data: compare timing without assuming causation



Synthetic p95 rises from 110 ms to 880 ms

A fictional deployment occurs at minute 3

A fictional human mitigation occurs at minute 7

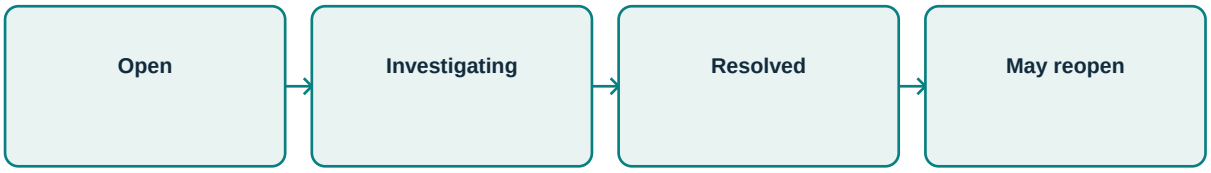
This invented p95 series illustrates how a timing view can guide questions during an investigation. The values are 110, 115, 118, 150, 420, 790, 880, 720, 410, 200, 135, and 120 milliseconds. A deployment marker at minute 3 and a human mitigation marker at minute 7 are scenario annotations. They do not establish that a deployment caused the spike or that the mitigation fixed it.

In the implemented overview, latency comes from numeric log metadata.latency_ms, with monitor-check latency used when there are no log latencies. The dashboard can aggregate a bounded sample, so its percentile should be read with the sampling flag and selected window. This graph is a teaching dataset rather than a dashboard export or a capacity benchmark.

Implementation reference: docs/architecture/system-design.md

Incident lifecycle

State changes make the investigation visible



Conceptual flow; the accompanying text defines exact relationships and optional paths.

Open: a new rule match or manual incident

Investigating: an operator is examining evidence

Resolved: an operator records closure; reopening is allowed

Incident records contain service, severity, status, rule fingerprint, summary, timestamps, and the latest analysis. The timeline records creation, status changes, and analysis activity; the audit table provides a broader system activity record. Operators can create incidents manually as well as receive rule-generated ones.

The statuses support coordination, but the application does not enforce a complex incident-management process, ownership system, or escalation schedule. Resolution is a human update even if a monitor becomes healthy. Reopening is allowed. Teams should record the actual causal finding and mitigation in their operational process rather than treating a generated probable cause as a completed postmortem.

Implementation reference: docs/architecture/system-design.md

Evidence-driven analysis

A bounded evidence set makes the explanation inspectable

Up to 30 service logs around the incident

Up to 3 prior deployments within two hours

Summary, probable cause, confidence, evidence, actions

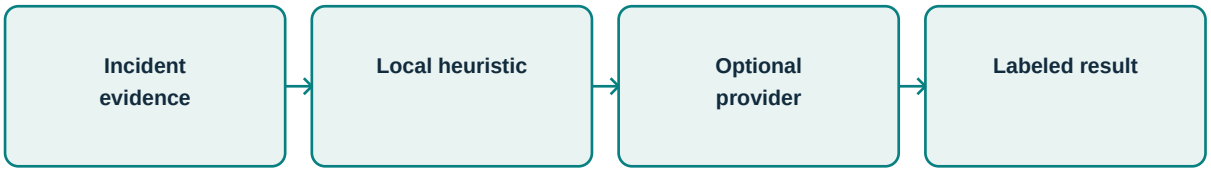
The analyzer selects logs for the exact incident service from 30 minutes before to one hour after incident creation. It includes at most 30 records, plus up to three deployments from the prior two hours. This bounds the context and links analysis to recorded evidence. Missing telemetry, inconsistent service names, delayed ingestion, and retention can all reduce what is available.

The local heuristic analyzer recognizes a small set of patterns, including connection pressure, timeouts, and rate limiting. Otherwise it can report insufficient evidence. The result includes a source and generation time. Confidence is an uncalibrated estimate, not a measured probability of correctness. A recent deployment is evidence to investigate, not proof of causality.

Implementation reference: docs/architecture/system-design.md

Optional AI and safe fallback

The application stays usable without a provider key



Conceptual flow; the accompanying text defines exact relationships and optional paths.

No provider key: local heuristic analysis

Provider configured: structured Responses API output

Provider failure or unusable output: labeled heuristic result

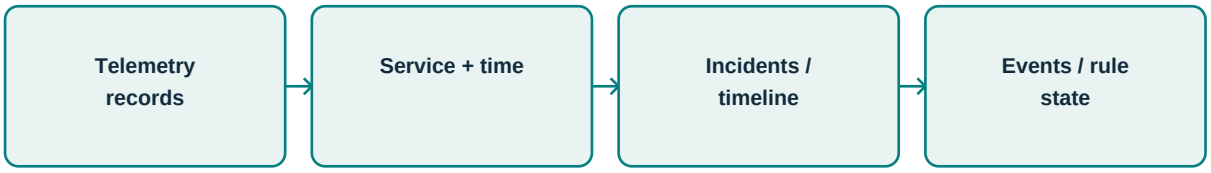
When OPENAI_API_KEY is configured, automatic worker analysis and manual analysis may send incident context and correlated evidence to the configured OpenAI model. Requests use a structured result schema and store=False. The source field distinguishes provider output from heuristic output; operators should inspect it before interpreting an investigation.

Telemetry is treated as untrusted prompt content. The analyzer has no infrastructure tools and cannot execute commands or rollbacks. Redaction is best effort, and store=False is a request setting rather than a guarantee about all provider retention. Only enable the integration after deciding which data may leave the system. Reanalysis replaces the latest analysis; older full outputs are not versioned, although timeline activity remains.

Implementation reference: docs/architecture/system-design.md

Persistent data model

Telemetry, investigation state, and work coordination



Conceptual flow; the accompanying text defines exact relationships and optional paths.

Telemetry: monitors, monitor_checks, logs, deployments

Investigation: incidents, incident_timeline, audit

Coordination: events and rule_states

The database stores UUID-string identifiers and UTC timestamps. A monitor has many checks by monitor ID. Incident timelines reference incident IDs. Logs and deployments correlate with incidents by exact service name and time windows, rather than a full tracing graph. Events contain durable payloads, availability, attempt count, lease state, and processed status.

These are logical relationships; the current schema does not enforce database foreign-key cascades. Deleting a monitor explicitly deletes its checks. SQLite uses WAL and a busy timeout for local work. PostgreSQL is the Compose database. The API and worker must share the same database URL, including the working directory when a relative SQLite path is used.

Implementation reference: apps/api/app/models.py

Durability and delivery

A recoverable SQL outbox with optional Redis transport

Conditional claim with a two-minute lease

Up to 50 events and 20 due monitors per tick

Five attempts, exponential retries, then event.failed

The worker conditionally claims an event and increments its attempt count before handling it. Database side effects and success are committed together. If the worker crashes, another worker can recover the event after the lease expires. Redis uses the tracehunt:events stream and workers consumer group to transport identifiers. The SQL sweep continues even when Redis fails.

Retries use exponential delays capped at 300 seconds. After five attempts, an event is marked processed and an event.failed audit entry is recorded; there is no dead-letter queue or replay endpoint. HTTP webhooks are at least once: a receiver may accept a request before the worker commits. Receivers must deduplicate by event_id. The two-second tick wait is not a promise of two-second end-to-end detection.

Implementation reference: apps/api/app/worker.py

Dashboard and metrics

Read each number according to what it actually measures

Overview counts ingested logs, not independent HTTP traffic

Uptime counts checks matching the expected status

Charts and percentiles may use a bounded sample

The overview supports windows from one to 168 hours, with 24 time buckets. Its request count is a count of ingested log rows; error rate is the fraction with ERROR/FATAL severity. Latency uses log metadata or monitor checks. Total log and error counts are separate database counts, but charts, services, and percentiles load at most 50,000 records. The sampled flag signals when the log limit is exceeded.

Monitor-list uptime uses the last 24 hours. Slow status-matching checks count as successful, and no checks displays 100%, which is not observed availability. The dashboard refreshes via API requests; the separate SSE endpoint polls SQL audit records every three seconds. The four exposed metrics are database gauges; logs_total is retained-row count rather than a monotonically increasing counter.

Implementation reference: docs/architecture/system-design.md

Local setup and Compose

Two supported paths share the same application model

Compose: web, API, worker, PostgreSQL, Redis

Local: Python API/worker, SQLite, Next.js frontend

Apply migrations before starting a new database

For Compose, copy .env.example to .env, review it, then run docker compose up --build -d --wait. The configured dashboard is localhost:3000 and the API is localhost:8000. The API applies alembic upgrade head before starting, and the worker follows API readiness. Docker needs a running Linux container engine. The current development session used frontend port 3001; that is separate from the Compose default.

For local Python, install the API requirements, work from apps/api, run alembic upgrade head, and start uvicorn app.main:app --host 127.0.0.1 --port 8000. Run python -m app.worker separately with the same environment. Install frontend dependencies and use the README web command. Do not blindly migrate or stamp an existing demo-created database; preserve it and follow the runbook adoption guidance.

Implementation reference: docs/modules/operations.md

API integration

Send structured records and query the resulting workflow

Base path /api/v1; interactive schema at /docs

Logs and deployments are POST ingestion endpoints

Monitor checks and incident analysis are explicit actions

Integrations send JSON to /api/v1/logs, /api/v1/logs/batch, and /api/v1/deployments. Read endpoints cover overview, monitors, logs, incidents, deployments, audit, and settings. A monitor can be created, edited, deleted, or checked manually. Incidents support creation, detail, status updates, and an analyze action. Consult the generated OpenAPI schema for exact request bodies and limits.

Outside demo mode include X-API-Key. A minimal log body is {"service":"payments","level":"ERROR","message":"upstream timeout","metadata":{"latency_ms":850}}. Repeated error logs can trigger a rule only when the worker is running and its current window meets the threshold. HTTP success from ingestion proves persistence, not that detection or an external alert has completed.

Implementation reference: docs/modules/api.md

Security boundaries

Use private deployment controls before handling sensitive data

Non-demo access uses one shared API key

Validate outbound destinations and restrict egress

Remove secrets before ingestion and before AI processing

Demo mode is unauthenticated and seeded with synthetic data. For a private deployment disable DEMO_MODE and SEED_DEMO, set a random API_KEY of at least 24 characters, and use unique database credentials. Add HTTPS, external authentication or a private network, gateway request/rate limits, and protected backups. CORS is an allowed-origin control, not a replacement for authorization.

There are no built-in users, tenant boundaries, SSO, or RBAC. The workspace key is stored in browser session storage. Monitor and webhook destinations share the private-network policy; enabling private access widens both. Webhooks do not have signatures or custom authorization headers. Regex redaction cannot guarantee confidentiality, so minimize input data and review provider data handling before enabling AI.

Implementation reference: SECURITY.md

Operating and recovering

Watch the worker and protect the SQL database

Readiness checks the database, not end-to-end health

Retention defaults to 30 days for selected record types

Test backups in an isolated environment before relying on them

A successful /ready checks database connectivity and the monitor table. It does not prove worker liveness, Redis health, webhook delivery, or full schema compatibility. If checks stop or pending events grow, inspect worker logs and confirm its database configuration. event.retry and event.failed audit records explain delivery trouble; exhausted events no longer count as pending.

Retention runs approximately hourly in the worker and removes old logs, checks, audit, and processed events. Incidents, timelines, deployments, rule states, and unprocessed events remain. SQL backups are authoritative; Redis is not a substitute. Stop SQLite writers or use its backup API. Restore into a separate database with workers and outbound integrations disabled until verified, because restored pending events may trigger external effects.

Implementation reference: docs/modules/operations.md

Validation and honest limits

Different checks establish different kinds of confidence

Recorded baseline: 46 API tests, Ruff, typecheck, build

Local smoke checks exercise API and worker workflows

No measured scale, availability, or AI accuracy guarantee

The implementation handoff recorded 46 passing backend tests, a clean Ruff check, passing TypeScript and production build checks, and ten live smoke checks. These are a baseline from the implementation work, not fresh results produced by this documentation build. Re-run the repository commands and inspect CI for the revision being deployed. The optional Compose test profile probes a controlled nginx endpoint.

The local Docker engine was unavailable during the prior validation, and no live provider-quality evaluation was established. Unit tests and smoke checks do not prove sustained throughput, multi-worker correctness under all failures, reliable Internet egress, or production availability. Useful next tests include queue delay under load, dependency failures, restore exercises, and an evaluated incident dataset for analysis quality.

Implementation reference: README.md

Worked incident example

An engineer follows the evidence instead of trusting a guess

A payments release is recorded and errors increase

A rule opens an incident and gathers nearby evidence

The engineer verifies the hypothesis and resolves manually

In this fictional scenario a payments deployment is recorded at minute 3. At minute 4, a five-minute error count reaches nine and creates a high incident. The worker gathers service logs and the nearby deployment. Suppose logs mention connection pool exhaustion: the heuristic may suggest connection pressure, while optional AI may organize the same evidence into an explanation. Neither proves that the release caused the issue.

The engineer checks the real database pool, request behavior, release diff, and upstream dependencies. A mitigation happens outside the app at minute 7. Lower subsequent errors support recovery but do not close the incident automatically. The engineer checks monitor results, records the outcome, and resolves it. This scenario ties the synthetic graphs to a human investigation, without claiming a measured recovery time.

Implementation reference: docs/architecture/system-design.md

Roadmap and source map

Extend the foundation with evidence and explicit priorities

Identity, permissions, signed alerts, and stronger audit

Rollups, worker health, replay tooling, and load testing

Tracing ingestion and evaluated adaptive detection

The listed extensions are roadmap ideas, not implemented features. Identity and role controls would improve accountability. Signed notification delivery and replay tooling would improve operations. Rollups and a search strategy would address growth beyond bounded scans. Worker health signals, queue-age metrics, restore tests, and load tests would make deployment limits measurable. OpenTelemetry ingestion and adaptive detection need new data models and evaluation.

Start with README.md for setup, docs/architecture/system-design.md for semantics, docs/modules/api.md for endpoint contracts, docs/modules/operations.md for operations, and SECURITY.md for boundaries. Implementation references are pinned to revision 0f148d696d546a78c850bdffa3a55eb52c56ce6b. This documentation separates current behavior, illustrative scenarios, and future work so the project can be explained accurately to engineers, reviewers, and collaborators.

Implementation reference: README.md